

Pygame Tiled Map JSON Integration

<https://gemini.google.com/app/9567b0395a99095f>

User prompt: how would i make my json map work in pygame if my json file looks like this: { "compressionlevel":-1, "height":33, "infinite":false, "layers":[{"id":2, "image":".VbaseVmain.png", "imageheight":1073, "imagewidth":1615, "name":"cave 1", "opacity":1, "type":"imagelayer", "visible":true, "x":0, "y":0, {"class":"collision", "draworder":"topdown", "id":5, "name":"wall_collisions", "objects":[{"height":0, "id":38, "name":""," "opacity":1, "polygon":[{"x":0, "y":0, {"x":43.85, "y":17.54, {"x":57.005, "y":43.85, {"x":74.5451, "y":37.2725, {"x":111.818, "y":74.5451, {"x":138.128, "y":74.5451, {"x":157.86, "y":89.8926, {"x":186.363, "y":61.39, {"x":230.213, "y":129.358, {"x":234.598, "y":188.555, {"x":197.325, "y":201.71, {"x":184.17, "y":234.598, {"x":155.668, "y":282.833, {"x":100.855, "y":302.565, {"x":37.2725, "y":302.565, {"x":32.8875, "y":298.18, {"x":59.1975, "y":271.87, {"x":76.7376, "y":245.56, {"x":107.433, "y":236.79, {"x":120.588, "y":203.903, {"x":129.358, "y":162.245, {"x":133.743, "y":131.55, {"x":103.048, "y":92.0851, {"x":89.8926, "y":65.775, {"x":54.8125, "y":54.8125, {"x":76.7376, "y":43.85, {"x":111.818, "y":50.4275, {"x":118.395, "y":100.855, {"x":48.235, "y":94.2776, {"x":43.85, "y":37.2725}], "rotation":0, "type":""," "visible":true, "width":0, "x":914.273, "y":366.148, {"height":0, "id":39, "name":""," "opacity":1, "polygon":[{"x":0, "y":0, {"x":96.4701, "y":4.385, {"x":94.2776, "y":192.94, {"x":17.54, "y":146.898}], "rotation":0, "type":""," "visible":true, "width":0, "x":982.241, "y":236.79, {"height":0, "id":40, "name":""," "opacity":1, "polygon":[{"x":0, "y":0, {"x":39.465, "y":2.1925, {"x":74.5451, "y":39.465, {"x":153.475, "y":35.08, {"x":184.17, "y":0, {"x":199.518, "y":52.62, {"x":157.86, "y":122.78, {"x":37.2725, "y":236.79, {"x":15.3475, "y":223.635, {"x":17.54, "y":210.48, {"x":26.31, "y":206.095, {"x":48.235, "y":140.32, {"x":83.3151, "y":111.818, {"x":89.8926, "y":81.1226, {"x":59.1975, "y":43.85, {"x":17.54, "y":37.2725}], "rotation":0, "type":""," "visible":true, "width":0, "x":1284.81, "y":684.061, {"height":0, "id":41, "name":""," "opacity":1, "polygon":[{"x":2.1925, "y":13.155, {"x":4.385, "y":48.235, {"x":46.0425, "y":83.3151, {"x":37.2725, "y":166.63, {"x":94.2776, "y":197.325, {"x":94.2776, "y":293.795, {"x":54.8125, "y":295.988, {"x":65.775, "y":228.02, {"x":98.6626, "y":219.25, {"x":118.395, "y":238.983, {"x":116.203, "y":274.063, {"x":142.513, "y":278.448, {"x":144.705, "y":488.928, {"x":87.7001, "y":532.778, {"x":85.5076, "y":642.403, {"x":48.235, "y":666.52, {"x":46.0425, "y":806.841, {"x":46.0425, "y":804.648, {"x":74.5451, "y":767.376, {"x":81.1226, "y":502.083, {"x":114.01, "y":491.12, {"x":116.203, "y":359.57, {"x":146.898, "y":348.608, {"x":146.898, "y":291.603, {"x":173.208, "y":285.025, {"x":171.015, "y":210.48, {"x":225.828, "y":210.48, {"x":243.368, "y":131.55, {"x":285.025, "y":120.588, {"x":309.143, "y":122.78, {"x":313.528, "y":164.438, {"x":361.763, "y":157.86, {"x":368.34, "y":122.78, {"x":418.768, "y":111.818, {"x":453.848, "y":85.5076, {"x":486.735, "y":59.1975, {"x":515.238, "y":59.1975, {"x":554.703, "y":32.8875, {"x":594.168, "y":28.5025, {"x":633.633, "y":48.235, {"x":655.558, "y":61.39, {"x":716.948, "y":54.8125, {"x":736.681, "y":61.39, {"x":741.066, "y":89.8926, {"x":749.836, "y":124.973, {"x":780.531, "y":142.513, {"x":798.071, "y":190.748, {"x":795.878, "y":236.79, {"x":830.958, "y":254.33, {"x":861.653, "y":254.33, {"x":881.386, "y":234.598, {"x":870.423, "y":151.283, {"x":942.776, "y":122.78, {"x":962.508, "y":65.775, {"x":1076.52, "y":43.85, {"x":1157.64, "y":87.7001, {"x":1159.83, "y":135.935, {"x":1164.22, "y":230.213, {"x":1225.61, "y":236.79, {"x":1225.61, "y":331.068, {"x":1258.5, "y":348.608, {"x":1317.69, "y":350.8, {"x":1308.92, "y":26.31}], "rotation":0, "type":""," "visible":true, "width":0, "x":324.49, "y":1061.17, {"height":0, "id":42, "name":""," "opacity":1, "polygon":[{"x":0, "y":0, {"x":54.8125, "y":8.77001, {"x":61.39, "y":129.358, {"x":26.31, "y":184.17, {"x":15.3475, "y":208.288, {"x":10.9625, "y":234.598, {"x":21.925, "y":236.79, {"x":35.08, "y":230.213, {"x":30.695, "y":342.03, {"x":78.9301, "y":414.383, {"x":57.005, "y":477.965, {"x":21.925, "y":513.045, {"x":26.31, "y":519.623, {"x":59.1975, "y":530.585, {"x":70.1601, "y":559.088, {"x":28.5025, "y":576.628, {"x":13.155, "y":567.858, {"x":17.54, "y":532.778, {"x":30.695, "y":519.623, {"x":81.1226, "y":460.425, {"x":89.8926, "y":403.42, {"x":70.1601, "y":328.875, {"x":39.465, "y":304.758, {"x":39.465, "y":258.715, {"x":74.5451, "y":267.485, {"x":87.7001, "y":256.523, {"x":83.3151, "y":230.213, {"x":39.465, "y":221.443, {"x":46.0425, "y":43.85}], "rotation":0, "type":""," "visible":true, "width":0, "x":644.595, "y":256.523, {"height":0, "id":43, "name":""," "opacity":1, "polygon":[{"x":0, "y":0, {"x":6.5775, "y":70.1601, {"x":37.2725, "y":81.1226, {"x":28.5025, "y":175.4, {"x":72.3526, "y":206.095, {"x":107.433, "y":234.598, {"x":127.165, "y":315.72, {"x":153.475, "y":339.838, {"x":149.09, "y":409.998, {"x":177.593, "y":414.383, {"x":173.208, "y":526.2, {"x":109.625, "y":576.628, {"x":81.1226, "y":609.515, {"x":85.5076, "y":710.371, {"x":120.588, "y":730.103, {"x":129.358, "y":789.301, {"x":162.245, "y":795.878, {"x":155.668, "y":887.963, {"x":120.588, "y":914.273, {"x":35.08, "y":927.428, {"x":50.4275, "y":983.578, {"x":81.1226, "y":861.653, {"x":127.165, "y":879.193, {"x":131.55, "y":929.621, {"x":188.555, "y":944.968, {"x":243.368, "y":951.546, {"x":276.255, "y":925.236, {"x":293.795, "y":890.156, {"x":469.195, "y":894.541, {"x":499.89, "y":916.466, {"x":543.74, "y":916.466, {"x":581.013, "y":890.156, {"x":644.595, "y":901.118, {"x":730.103, "y":892.348, {"x":769.568, "y":905.503, {"x":795.878, "y":936.198, {"x":879.193, "y":940.583, {"x":951.546, "y":934.006, {"x":962.508, "y":844.113, {"x":988.818, "y":762.991, {"x":1050.21, "y":749.836, {"x":1098.44, "y":793.686, {"x":1115.98, "y":931.813, {"x":1240.96, "y":929.621, {"x":1249.73, "y":870.423, {"x":1295.77, "y":868.231, {"x":1302.35, "y":804.648, {"x":1251.92, "y":817.803, {"x":1210.26, "y":784.916, {"x":1203.68, "y":648.98, {"x":1297.96, "y":528.393, {"x":1315.5, "y":484.543, {"x":1337.43, "y":473.58, {"x":1337.43, "y":133.43, "y":352.993, {"x":1466.78, "y":368.34, {"x":1460.21, "y":1131.33, {"x":1263.1, "y":1133.52, {"x":361.763, "y":1113.79, {"x":265.293, "y":10.9625}], "rotation":0, "type":""," "visible":true, "width":0, "x":223.635, "y":1078.71, {"height":0, "id":44, "name":""," "opacity":1, "polygon":[{"x":0, "y":0, {"x":6.5775, "y":74.5451, {"x":32.8875, "y":107.433, {"x":48.235, "y":151.283, {"x":72.3526, "y":157.86, {"x":74.5451, "y":96.4701, {"x":54.8125, "y":81.1226, {"x":48.235, "y":24.1175}], "rotation":0, "type":""," "visible":true, "width":0, "x":1280.42, "y":473.58, {"height":0, "id":45, "name":""," "opacity":1, "polygon":[{"x":0, "y":0, {"x":2.1925, "y":63.5825, {"x":48.235, "y":89.8926, {"x":59.1975, "y":4.385}], "rotation":0, "type":""," "visible":true, "width":0, "x":1385.66, "y":317.913}], "opacity":1, "type":"objectgroup", "visible":true, "x":0, "y":0 }, {"nextlayerid":8, "nextobjectid":46, "orientation":"orthogonal", "renderorder":"right-down", "tiledversion":"1.12.2", "tileheight":32, "tilessets":[{"firstgid":1, "source":".VactiveVmining_nodes_32x.tsx"}], "tilewidth":32, "type":"map", "version":"1.10", "width":50 }

Response: Your JSON file is exported from **Tiled Map Editor**. Instead of using grid-based tiles for the visuals, this map relies on a single large image layer ("imagelayer") for the background artwork and an object layer ("objectgroup") containing custom vector polygon paths for your wall collisions.

To make this work in Pygame, you need to do two main things:

1. Load and display the background image.
2. Parse the polygon coordinates, converting them from Tiled's **relative system** to Pygame's **absolute screen system**, so you can use them for collisions.

Here is a complete script demonstrating how to load your specific JSON data and render it.

The Pygame Implementation

```
import json
import pygame
import sys

# Initialize Pygame
pygame.init()

# Setup screen sizes matching your JSON's imagewidth (1615) and imageheight (1073)
SCREEN_WIDTH = 1615
SCREEN_HEIGHT = 1073
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("Loading Tiled JSON Map")
clock = pygame.time.Clock()

# -----
# 1. LOAD AND PARSE THE JSON FILE
# -----
# Replace 'map.json' with the actual path to your saved JSON file
with open("map.json", "r") as file:
    map_data = json.load(file)

background_image = None
collision_polygons = []

# Loop through the layers defined in your JSON
for layer in map_data["layers"]:

    # Handle the artwork layer
    if layer["type"] == "imagelayer":
        # Tiled uses relative paths like "../base/main.png"
        # Adjust this string to match where the image lives relative to this Python script
        image_path = "base/main.png"
        try:
            background_image = pygame.image.load(image_path).convert_alpha()
        except pygame.error as e:
            print(f"Could not load image at {image_path}. Check your folders! Error: {e}")

    # Handle the collision layer
    elif layer["type"] == "objectgroup" and layer["name"] == "wall_collisions":
        for obj in layer["objects"]:
            if "polygon" in obj:
                # The object's base position in the world
                base_x = obj["x"]
                base_y = obj["y"]

                # Reconstruct the absolute coordinates for every vertex
                absolute_points = []
                for vertex in obj["polygon"]:
                    world_x = base_x + vertex["x"]
                    world_y = base_y + vertex["y"]
                    absolute_points.append((world_x, world_y))

                collision_polygons.append(absolute_points)
```

```

# -----
# MAIN GAME LOOP
# -----
running = True
show_debug_walls = True # Toggle this to hide/show the red collision lines

while running:
    clock.tick(60)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_d: # Press 'D' to toggle collision visibility
                show_debug_walls = not show_debug_walls

    # Draw the background image layer
    if background_image:
        screen.blit(background_image, (0, 0))
    else:
        screen.fill((30, 30, 30)) # Fallback dark background

    # Draw the collision lines for testing
    if show_debug_walls:
        for polygon in collision_polygons:
            # Draws lines connecting the points. True closes the loop back to the start.
            pygame.draw.lines(screen, (255, 0, 0), True, polygon, 3)

    pygame.display.flip()

pygame.quit()
sys.exit()

```

How the Coordinate Math Works

Tiled saves space by making polygon vertex positions relative to the overall object's starting coordinate point. If you tried to pass the raw polygon coordinates straight into Pygame, your walls would group up erroneously in the top-left corner of the screen.

To fix this, the script translates the positions using this conversion:

$$\text{Absolute Point} = (x_{\text{base}} + x_{\text{vertex}}, y_{\text{base}} + y_{\text{vertex}})$$

- $x_{\text{base}}, y_{\text{base}}$: The starting world point of the object (e.g., 914.273, 366.148 from your first object).
- $x_{\text{vertex}}, y_{\text{vertex}}$: The offset values inside the "polygon" array list.

Pro-Tip: Handling Player Collisions against Polygons

Standard Pygame Rects (`pygame.Rect`) only handle AABB (Axis-Aligned Bounding Box) shapes, meaning they can't natively check if a moving player has crossed an angled vector line. To achieve collisions with these custom shapes easily without heavy math libraries, you can build a **Collision Mask**:

1. Create a transparent surface the exact size of your map:

```
collision_surface = pygame.Surface((SCREEN_WIDTH, SCREEN_HEIGHT), pygame.SRCALPHA)
```

2. Draw your parsed polygons filled with a solid color onto that surface:

```
for polygon in collision_polygons:
    pygame.draw.polygon(collision_surface, (255, 255, 255, 255), polygon)
```

3. Generate a mask from that surface:

```
map_mask = pygame.mask.from_surface(collision_surface)
```

Now, you can turn your player's sprite into a mask and use `map_mask.overlap(player_mask, offset)` to achieve pixel-perfect wall collision detection against all your custom cave geometry instantly!
